REPORTE DE PRÁCTICA: FESTIVAL DE MÚSICA

Tema: Herencia y Polimorfismo en Programación Orientada a Objetos

Estudiante: Joaquín García
Institución: Tecnológico Nacional de México — Instituto Tecnológico de Aguascalientes
Materia: Programación Orientada a Objetos | Lenguaje: Python 3
Repositorio GitHub: https://github.com/joaquinrg132-png/festival-musica-poo

1. Objetivo de la Actividad

El objetivo es diseñar e implementar un programa orientado a objetos en Python que simule un festival musical con diversos tipos de artistas (Cantante, DJ, Banda), aplicando rigurosamente los conceptos de Herencia, Polimorfismo y constructores para permitir que cada participante se presente, actúe y se despida con su propia identidad.

2. Fundamentación Teórica y Arquitectura

• Herencia (Clase Base y Subclases):
Se implementó la clase base 'Artista' que encapsula los atributos comunes: nombre, género y popularidad (1 a 100), así como los métodos presentarse(), actuar() y despedirse(). Las subclases 'Cantante', 'DJ' y 'Banda' extienden de Artista y utilizan super().__init__(nombre, genero, popularidad) para reutilizar la lógica de inicialización, agregando atributos específicos como cancion_mas_popular, estilo e integrantes.

• Polimorfismo y Sobrescritura de Métodos:
Cada subclase sobrescribe el método actuar() adaptando su comportamiento particular. En la función iniciar_festival(lista_artistas), se itera sobre la colección invocando presentarse(), actuar() y despedirse() de forma uniforme. Esto cumple con el principio de polimorfismo puro, eliminando por completo sentencias condicionales (if/elif) para discriminar el tipo de objeto en tiempo de ejecución.

🔗 **ENLACE OFICIAL AL REPOSITORIO DE GITHUB (PÚBLICO)**
https://github.com/joaquinrg132-png/festival-musica-poo

3. Evidencias de Ejecución del Programa

Capturas reales de la terminal interactiva demostrando el registro y la ejecución polimórfica:

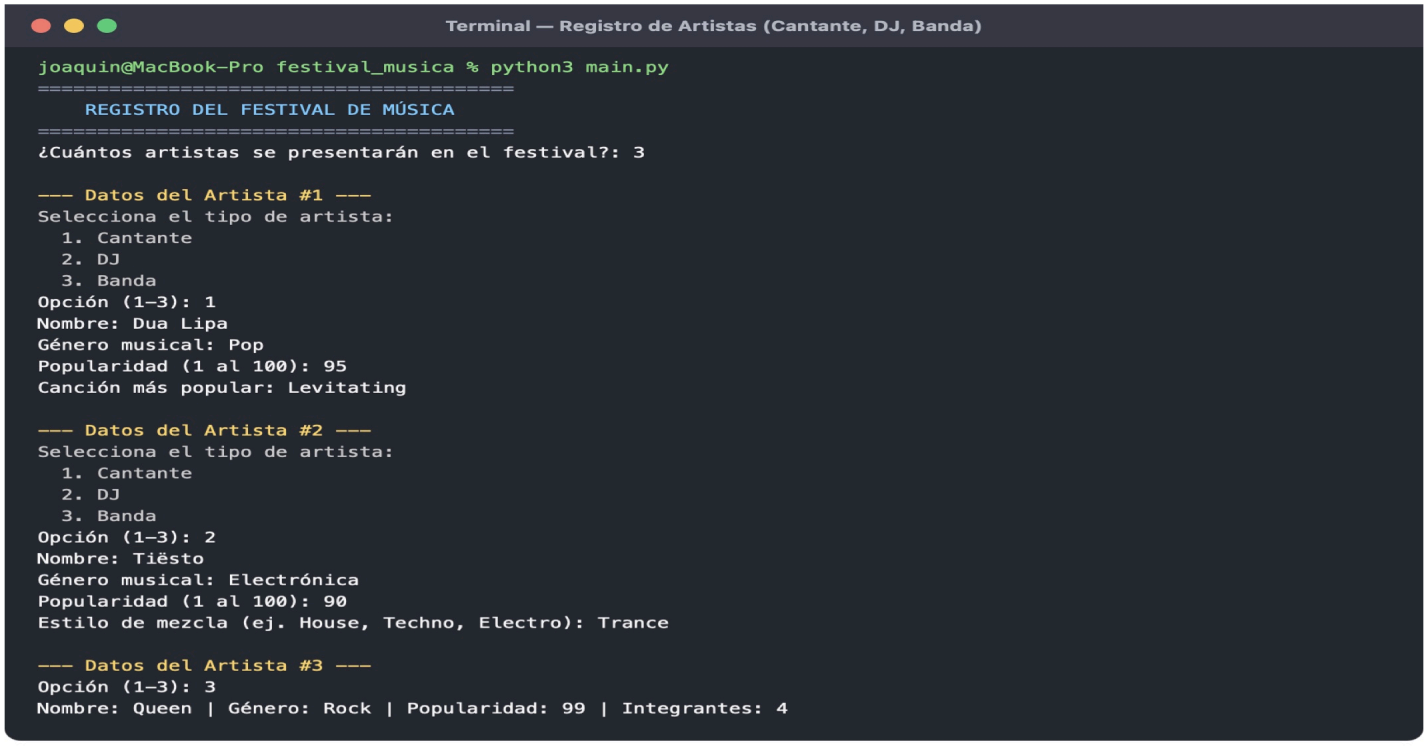


Figura 1: Registro interactivo de artistas solicitando nombre, género, popularidad (1-100) y atributos específicos.

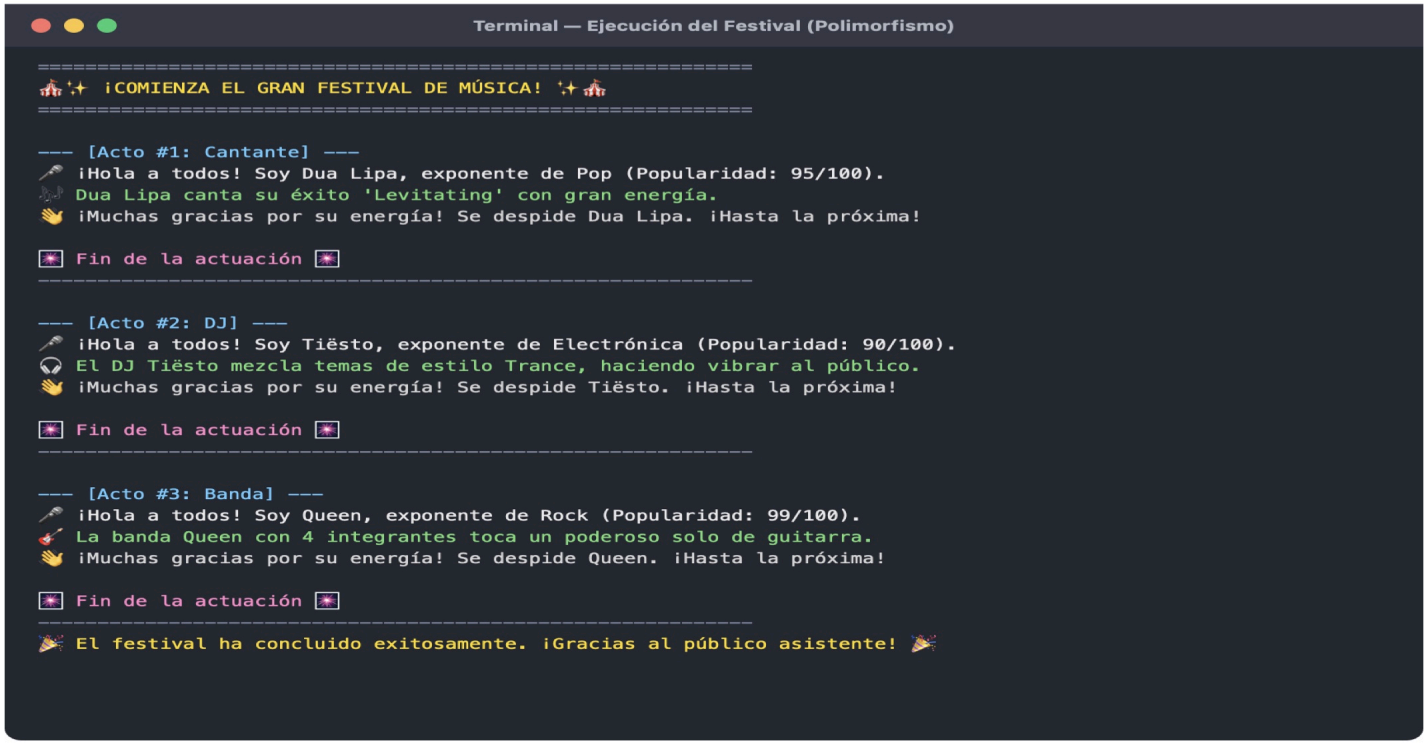


Figura 2: Invocación polimórfica de actuar() para Cantante, DJ y Banda con separador de actuaciones.

4. Código Fuente Completo (Parte 1: Clases y Herencia)

Definición de la clase base Artista y las subclases especializadas Cantante, DJ y Banda:

```
# ============================================================
# 1️⃣ CLASE BASE: Artista
# ============================================================
class Artista:
    """Clase base que representa a un artista musical genérico."""
    def __init__(self, nombre: str, genero: str, popularidad: int):
        self.nombre = nombre
        self.genero = genero
        self.popularidad = popularidad

    def presentarse(self):
        print(f"🎤 ¡Hola a todos! Soy {self.nombre}, exponente de {self.genero} (Popularidad: {self.popularidad}/100).")

    def actuar(self):
        """Método base que será sobrescrito por cada subclase (polimorfismo)."""
        print(f"🎵 {self.nombre} está realizando una presentación en el escenario.")

    def despedirse(self):
        print(f"👋 ¡Muchas gracias por su energía! Se despide {self.nombre}. ¡Hasta la próxima!")

# ============================================================
# 2️⃣ SUBCLASES DE Artista (HERENCIA Y POLIMORFISMO)
# ============================================================
class Cantante(Artista):
    """Subclase que representa a un cantante solista."""
    def __init__(self, nombre: str, genero: str, popularidad: int, cancion_mas_popular: str):
        super().__init__(nombre, genero, popularidad)  # Llamada al constructor padre
        self.cancion_mas_popular = cancion_mas_popular

    def actuar(self):
        print(f"🎶 {self.nombre} canta su éxito {self.cancion_mas_popular} con gran energía.")

class DJ(Artista):
    """Subclase que representa a un DJ."""
    def __init__(self, nombre: str, genero: str, popularidad: int, estilo: str):
        super().__init__(nombre, genero, popularidad)  # Llamada al constructor padre
        self.estilo = estilo

    def actuar(self):
        print(f"🎧 El DJ {self.nombre} mezcla temas de estilo {self.estilo}, haciendo vibrar al público.")

class Banda(Artista):
    """Subclase que representa a una banda musical."""
    def __init__(self, nombre: str, genero: str, popularidad: int, integrantes: int):
        super().__init__(nombre, genero, popularidad)  # Llamada al constructor padre
        self.integrantes = integrantes

    def actuar(self):
        print(f"🎸 La banda {self.nombre} con {self.integrantes} integrantes toca un poderoso solo de guitarra.")
```

4. Código Fuente (Parte 2: Función Polimórfica y main)

Implementación de iniciar_festival() y función interactiva de captura de datos:

```
# ============================================================
# 3️⃣ FUNCIÓN DE INICIO DEL FESTIVAL (POLIMORFISMO PURO)
# ============================================================
def iniciar_festival(lista_artistas):
    print("\n" + "=" * 60)
    print("🎪✨ ¡COMIENZA EL GRAN FESTIVAL DE MÚSICA! ✨🎪")
    print("=" * 60 + "\n")
    for i, artista in enumerate(lista_artistas, 1):
        print(f"--- [Acto #{i}] ---")
        artista.presentarse()
        artista.actuar()       # Invocación polimórfica: no requiere if/elif
        artista.despedirse()
        print("\n🎆 Fin de la actuación 🎆\n" + "-" * 60 + "\n")

# ============================================================
# 4️⃣ PROGRAMA PRINCIPAL (main.py)
# ============================================================
def pedir_entero_rango(mensaje: str, minimo: int, maximo: int) -> int:
    while True:
        try:
            val = int(input(mensaje))
            if minimo <= val <= maximo: return val
            print(f"⚠️ Ingresa un número entre {minimo} y {maximo}.")
        except ValueError:
            print("⚠️ Entrada no válida.")

def main():
    num = pedir_entero_rango("¿Cuántos artistas se presentarán?: ", 1, 50)
    lista_artistas = []
    for i in range(1, num + 1):
        tipo = pedir_entero_rango("Tipo (1. Cantante, 2. DJ, 3. Banda): ", 1, 3)
        nom, gen = input("Nombre: ").strip(), input("Género: ").strip()
        pop = pedir_entero_rango("Popularidad (1 al 100): ", 1, 100)
        if tipo == 1:
            lista_artistas.append(Cantante(nom, gen, pop, input("Canción más popular: ")))
        elif tipo == 2:
            lista_artistas.append(DJ(nom, gen, pop, input("Estilo: ")))
        else:
            lista_artistas.append(Banda(nom, gen, pop, pedir_entero_rango("Integrantes: ", 1, 50)))
    iniciar_festival(lista_artistas)

if __name__ == "__main__":
    main()
```

5. Conclusiones

La implementación demostró exitosamente cómo la herencia permite centralizar atributos y comportamientos comunes en la clase padre evitando duplicidad, mientras que el polimorfismo faculta al sistema a tratar a cualquier artista de manera homogénea sin acoplar la función principal a los tipos concretos de las subclases, logrando un código modular, extensible y limpio.